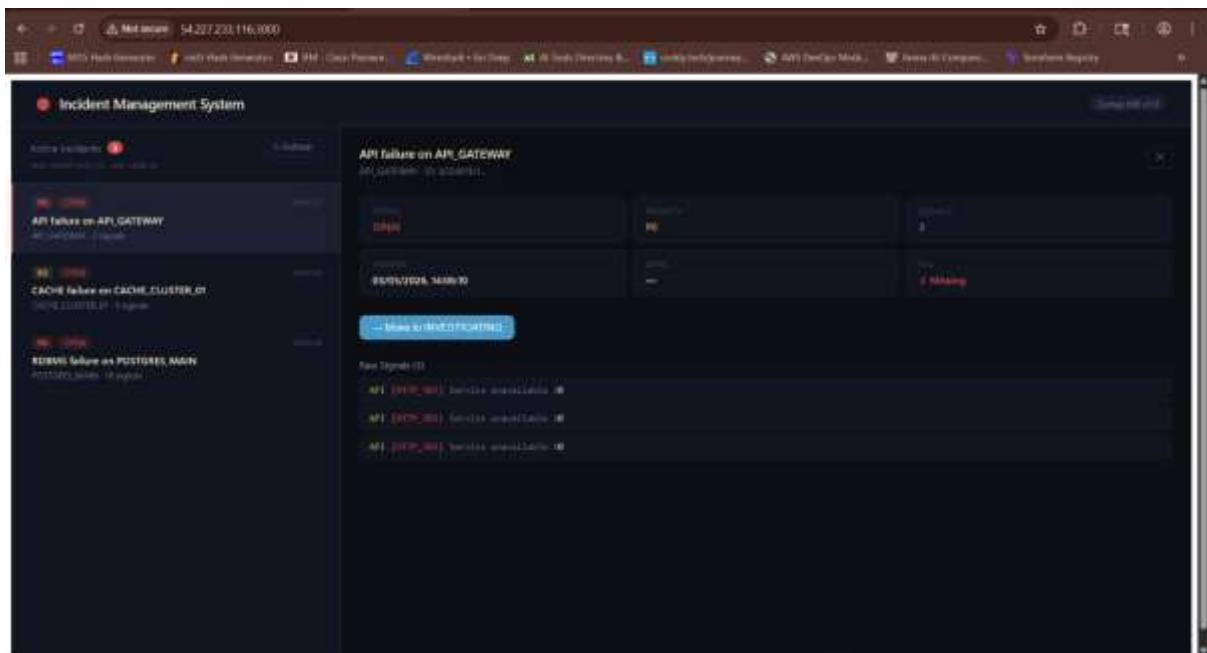


MHA Ramayya – Infrastructure / SRE Intern Assignment



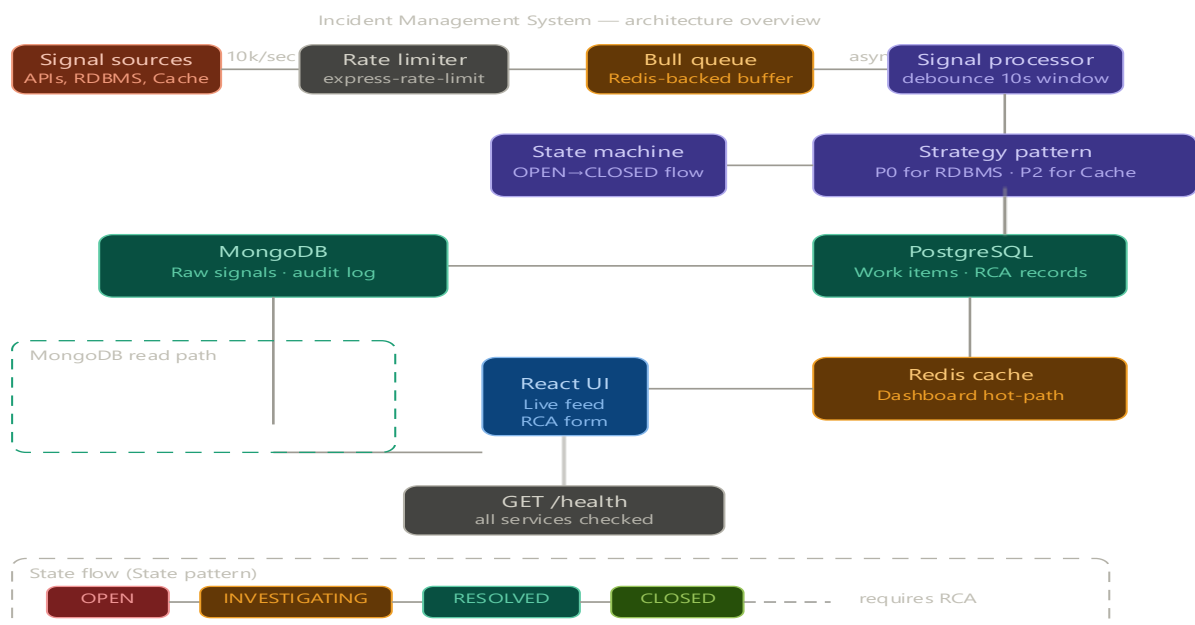
1. Project Overview

This project implements a **production-style Incident Management System (IMS)** with a complete DevOps lifecycle: CI/CD, monitoring, observability, and containerization. It demonstrates automation, resilience, and scalable deployment practices.

2. GitHub Repository

<https://github.com/babai-cyber/ims.git>

3. Architecture Diagram



Frontend (React + Vite) → Backend (Node.js + TypeScript + Express) → PostgreSQL (Work Items + RCA) + MongoDB (Raw Signals) + Redis (Cache + Queue) All services are containerized with **Docker Compose** and orchestrated via **Jenkins CI/CD**. Monitoring is powered by **Prometheus + Grafana**.

4. Architecture Explanation

- **Frontend:** React dashboard auto-refreshing every 5s, showing incidents and RCA forms.
- **Backend:** Node.js API with Bull queue for backpressure, Redis debounce logic, PostgreSQL for transactional state, MongoDB for raw signals.
- **Databases:** PostgreSQL (ACID transactions), MongoDB (TTL audit log), Redis (cache + queue).
- **Design Patterns:**
 - Strategy Pattern → P0 alerts for RDBMS/API, P2 alerts for Cache/Queue.
 - State Machine → Enforces OPEN → INVESTIGATING → RESOLVED → CLOSED lifecycle.

5. CI/CD Pipeline (Jenkins)

Before webhook trigger :

The screenshot displays the Jenkins pipeline for 'IMS'. The left sidebar contains navigation options like Status, Changes, Build Now, Configure, and Pipeline Syntax. The main area shows a table of pipeline stages and their durations for three builds.

Checkout	Install Node (if missing)	Install Dependencies	Backend Deps	Frontend Deps	SonarQube Analysis	Build Docker Images	Backend Image	Frontend Image	Trivy Scan	Push to ECR	Deploy	Declare Post Action
415ms	326ms	95ms	3s	1s	195ms	84ms	3s	4s	270ms	63ms	171ms	2s
6.25ms	36.2ms	177ms	3s	1s	228ms	80ms	10s	11s	660ms		223ms	8s
48.1ms	114ms	32ms	2s	1s	118ms	30ms	79ms	490ms	71ms	70ms	71ms	107ms
3.75ms	128ms	32ms	2s	1s	198ms	80ms	6.68ms	703ms	81ms	34ms	189ms	16.7s

Below the table, there are 'Permalinks' for the builds:

- Last build (PS) 35 sec ago
- Last stable build (PS) 35 sec ago
- Last successfully build (PS) 35 sec ago

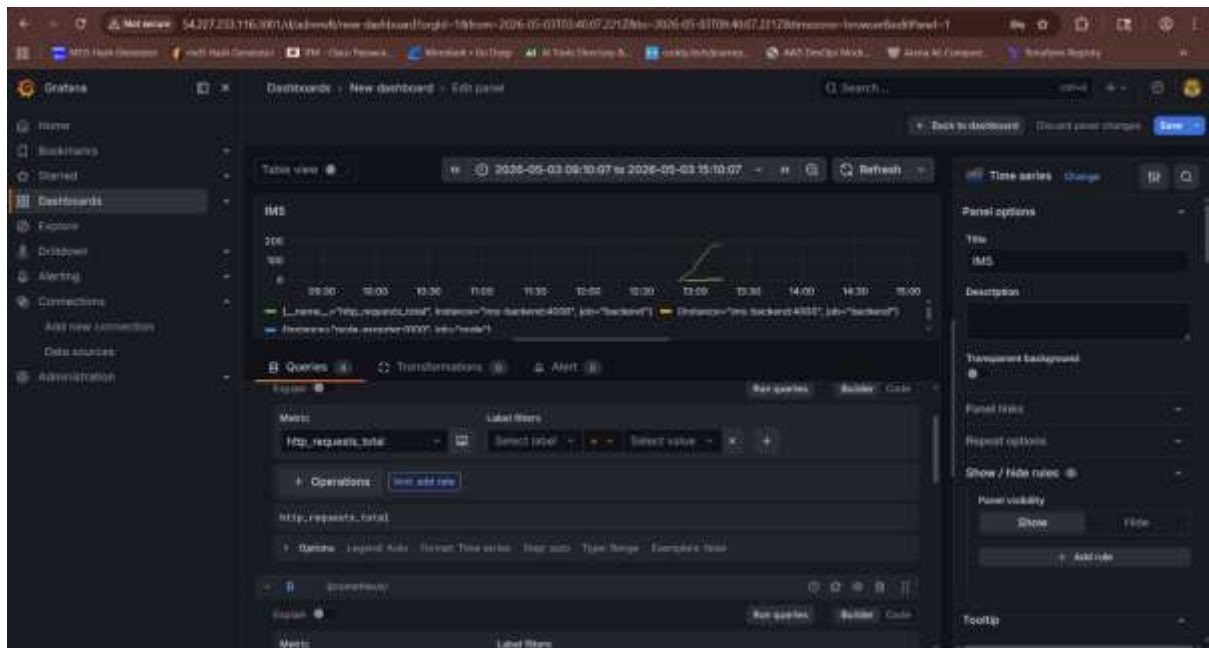


After Webhook trigger:

Stages:

1. **Checkout** (GitHub webhook) : <https://github.com/babai-cyber/ims.git>
2. **Install Dependencies** (npm ci)
3. **Build** (TypeScript compile)
4. **Unit Tests** (Jest for RCA/state machine/debounce)
5. **SonarQube Analysis** (Quality Gate enforcement)
6. **Trivy Scan** (Docker image CVE scanning)
7. **Docker Build & Push** (Docker Hub/ECR)
8. **Deploy** (EC2 via docker-compose)
9. **Health Check** (/health endpoint)

6. Monitoring (Prometheus + Grafana)

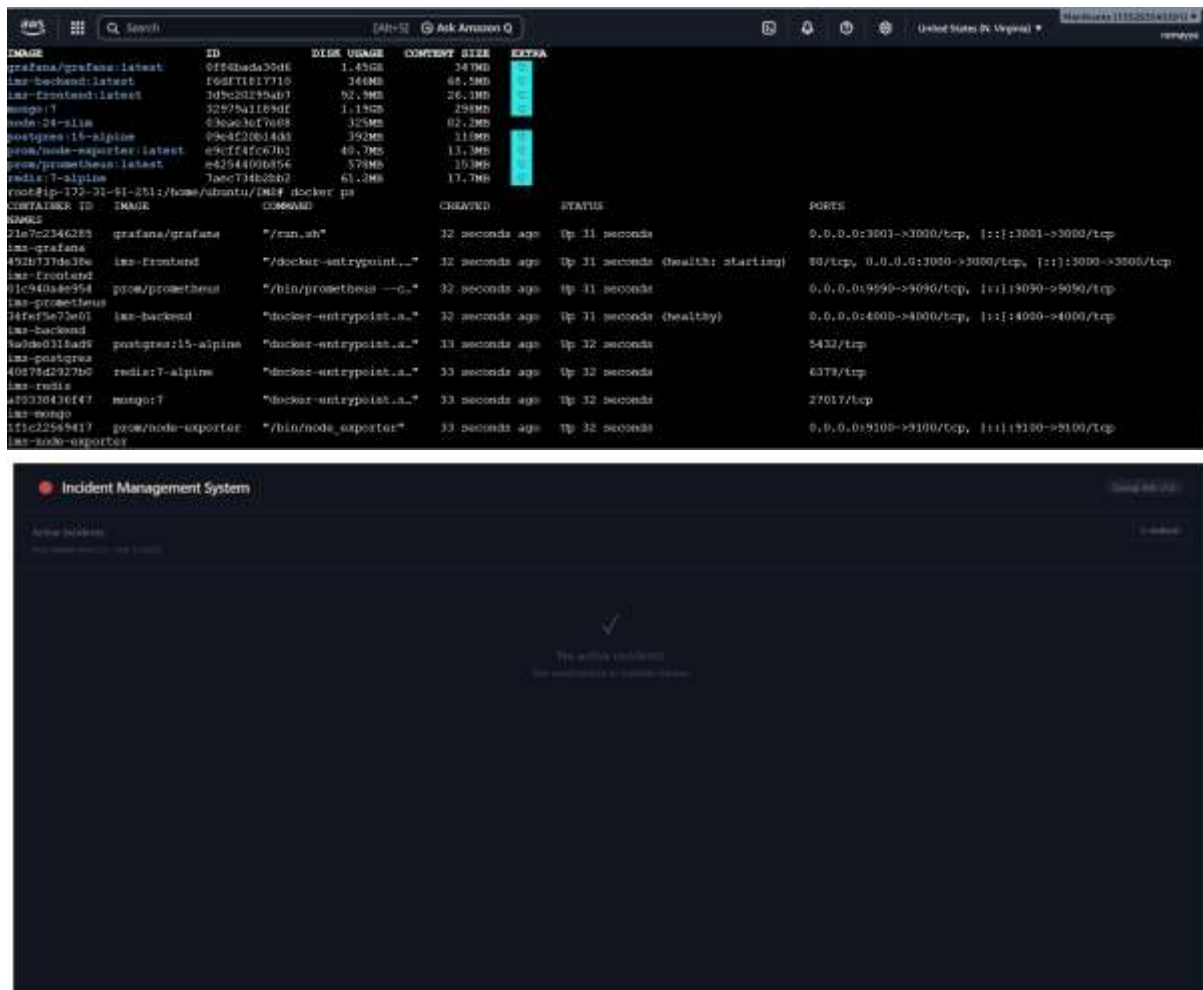


- **Prometheus** scrapes metrics every 15s from backend and node-exporter.
- **Grafana Dashboard Panels:**
 - Signal Throughput (rate of signals/sec)
 - Queue Depth (Bull jobs waiting)
 - HTTP Latency (P99 histogram)
 - Open Incidents Count

7. Security & Best Practices

- **Rate Limiting:** 10k requests/min per IP.
- **Input Validation:** Required fields enforced at API level.
- **CORS + Helmet.js:** Secure headers, restricted origins.
- **Secrets Management:** Environment variables + Jenkins credentials store.
- **Trivy CVE Scanning:** Fail pipeline on CRITICAL vulnerabilities.
- **SonarQube SAST:** No blockers/critical hotspots.
- **MongoDB TTL Index:** Auto-delete signals after 90 days.
- **AWS Security Groups:** Only required ports exposed.

8. Deployment



- **Local:** docker-compose up --build
- **Cloud:** AWS EC2 (t3.medium) .
- **Health Check:** curl http://<ec2-ip>:4000/health
- **Frontend:** http://<ec2-ip>:3000
- **Monitoring:** Prometheus (:9090), Grafana (:3001).

9. Screenshots (Attached in Submission) [link for project screenshots](#)

- Application UI (Incident Dashboard)
- Jenkins Pipeline Execution
- Grafana Dashboard
- Prometheus Targets
- SonarQube Quality Gate Report

10. Conclusion

This project demonstrates **SRE capabilities**:

- Handling **10k signals/sec** with backpressure.
- Enforcing **debounce logic** and RCA gating.
- Full **DevSecOps pipeline** with SonarQube + Trivy.
- **Observability stack** with Prometheus + Grafana.
- Secure, resilient, and scalable deployment on Docker + AWS EC2.